

Effective REST APIs Testing with Error Message Analysis

LIXIN XU, State Key Laboratory for Novel Software Technology, Nanjing University, China
HUAYAO WU*, State Key Laboratory for Novel Software Technology, Nanjing University, China
ZHENYU PAN, State Key Laboratory for Novel Software Technology, Nanjing University, China
TONGTONG XU, Huawei, China
SHAOHUA WANG, Central University of Finance and Economics, China
XINTAO NIU, State Key Laboratory for Novel Software Technology, Nanjing University, China
CHANGHAI NIE, State Key Laboratory for Novel Software Technology, Nanjing University, China

REST APIs are essential for building modern enterprise systems, but effectively testing them remains challenging, particularly due to difficulties in inferring constraints from specifications. Current testing approaches typically use feedback from HTTP status codes to guide input generation. However, they overlook valuable information available in the accompanying error messages, reducing their effectiveness in exploring the APIs' input spaces. In this paper, we propose EmRest, a black-box testing approach that leverages error message analysis to enhance both valid and exceptional test input generation for REST APIs. For each operation under test, EmRest first identifies all possible value assignment strategies for each of its input parameters. It then repeatedly applies combinatorial testing to sample test inputs based on these strategies, and statistically analyzes the error messages (of 400-range status code) received to infer and exclude invalid combinations of value assignment strategies (i.e., constraints of the input space). Additionally, EmRest seeks to mutate valid value assignment strategies that are finally identified to generate test inputs for exceptional testing. The error messages (of 500-range status code) received are categorized to identify bug-prone operations, for which more testing resources are allocated. Our experimental results on 16 real-world REST APIs demonstrates the effectiveness of EmRest. It achieves higher operation coverage than state-of-the-art approaches in 50% of APIs, and detects 226 unique bugs undetected by other approaches.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: REST APIs, OpenAPI, Feedback-Driven Testing

ACM Reference Format:
Lixin Xu, Huayao Wu, Zhenyu Pan, Tongtong Xu, Shaohua Wang, Xintao Niu, and Changhai Nie. 2025. Effective REST APIs Testing with Error Message Analysis. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA087 (July 2025), 23 pages. https://doi.org/10.1145/3728964

*Corresponding author.

Authors' Contact Information: Lixin Xu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, lxxu@smail.nju.edu.cn; Huayao Wu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, hywu@nju.edu.cn; Zhenyu Pan, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, zypan@smail.nju.edu.cn; Tongtong Xu, Huawei, Hangzhou, China, xutongtong9@huawei.com; Shaohua Wang, Central University of Finance and Economics, Beijing, China, davidshwang@ieee.org; Xintao Niu, State Key Laboratory for Novel Software Technology, Nanjing University, Suzhou, China, niuxintao@nju.edu.cn; Changhai Nie, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, changhainie@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2994-970X/2025/7-ARTISSTA087
https://doi.org/10.1145/3728964

1 Introduction

Given the widespread adoption of REST APIs in building modern enterprise systems [26], there is clearly a need to thoroughly test the correctness of such APIs to ensure system quality. Currently, OpenAPI Specification (OAS) [8] is the most well-known standard in documenting REST APIs, in which the operations that can be performed, and their input parameters required are explicitly described. The availability of OAS has then motivated researchers to develop many black-box REST APIs testing approaches [14, 26, 35, 62]. However, effective testing of these APIs remains challenging, not only due to the huge input space to explore, but more critically because the complex *constraints* involved in REST APIs cannot be easily identified from the given OAS.

In order to cater for the constraints in REST APIs, some studies propose extending OAS to allow developers to describe constraints using formal languages [17, 43]. However, their applications require manual efforts and are hard to automate. To automatically extract constraints from OAS, where constraints are often described in the description field using natural language, some testing approaches, like RestCT [61] and NLPtoREST [32], apply Natural Language Processing (NLP). However, these approaches typically rely on predefined syntactic patterns [44] for constraint extraction, which might be limited to certain language styles. Moreover, in practice, developers may omit constraints entirely in parameter descriptions [11] or provide descriptions that are incorrect or outdated due to server changes, further limiting the effectiveness of NLP-based approaches.

In contrast to extracting constraints directly from OAS, many current testing approaches [16, 33, 37, 40–42] rely on feedback mechanisms to dynamically incorporate constraints when generating test inputs. Despite differences in concrete mechanisms (e.g., the producer-consumer relationship exploited in RESTler [16], the attention model trained in MINER [42], and the reinforcement learning utilized in ARAT-RL [33]), these approaches all share a similar idea to generate constraints-satisfying test inputs for REST APIs: parameter assignments leading to 200-range HTTP responses (valid inputs) are more likely to be reused, while those resulting in 400-range responses (constraint violations) become less likely to recur.

Although these feedback-driven testing approaches can reach more operations when testing REST APIs [35], they only rely on HTTP status codes to guide test input generation, which may result in substantial time costs to find constraint-satisfying inputs. This is because for operations with multiple input parameters, not *all* parameters are relevant to the 400-range status code triggered; but due to the lack of detailed feedback, these approaches could only treat all parameter assignments as equally responsible for the constraint violation. Worse more, in practice, each operation usually has multiple constraints among different input parameters, and violating any one of them will result in the same 400-range status code. In this case, adjusting inputs to satisfy one constraint may inadvertently violate others previously satisfied.

In addition to exploring valid input spaces (and revealing bugs triggered by such inputs) of REST APIs, it is also important to examine the ability of REST APIs on handling input values that mismatch the definition in specification, namely, exceptional testing [20]. Current testing approaches [33, 42, 59] typically mutate valid value assignments, hoping to use potentially invalid values to trigger bugs relevant to parameter validation codes, where a 400-range status code is expected, but a 500-range status code occurs. However, existing approaches usually apply mutation operators randomly, which may inadequately test operations more prone to parameter validation bugs, thus limiting bug detection effectiveness.

To achieve a more directed and effective exploration of REST API input spaces, in this paper, we propose EmRest, a novel black-box approach that leverages information in *error messages* included in HTTP response bodies to guide test input generation. Specifically, 400-range error messages typically describe the exact errors triggered, which can thus be exploited to infer constraints among

input parameters. While error messages of 500-range status code (also referred to as *bug messages* in this study) usually include bug localization information (e.g., stack traces), which can aid in categorizing detected bug.

Specifically, EmRest consists of an *Inferring Constraints* phase and an *Exceptional Testing* phase, primarily utilizing 400-range and 500-range error messages, respectively. In the *Inferring Constraints* phase, EmRest seeks to gradually construct and explore the valid input space of each operation (different operations are tested in an order that considers operation dependencies). For each given operation, EmRest first initializes its input space by identifying all possible value assignment strategies for every input parameter. Then, EmRest repeatedly applies combinatorial testing to sample test inputs from the currently valid input space constructed so far, systematically examining parameters interactions, and at the same time, statistically assess how different combinations of value assignment strategies contribute to the triggering of each distinct error message of 400-range status codes. Here, if a combination of value assignment strategies appears highly frequently in test inputs that yield the same 400-range error message, then EmRest will treat this combination as an *invalid* combination (i.e., a constraint of the input space), and thereby exclude it in future test inputs sampling iterations.

After constructing and exploring the valid input space, EmRest proceeds to the *Exceptional Testing* phase, generating potentially invalid test inputs to examine the exception handling ability of the API. Specifically, EmRest first mutates value assignment strategies sampled from the valid input space of each operation at an initial round. Then, EmRest categorizes error messages of 500-range status codes (i.e., bug messages) received to identify operations triggering the largest number of unique bugs. For complex operations, multiple specification violations can occur through applying different mutation operators on various parameters, each potentially associated with distinct parameter validation code and bug messages. Therefore, EmRest prioritizes testing resources towards operations more prone to parameter validation issues, aiming to uncover diverse validation-related bugs effectively.

To evaluate the effectiveness of EmRest, we conducted experiments on 16 real-world REST APIs [33, 61], and compared EmRest against six State-Of-The-Art (SOTA) REST APIs testing approaches: ARAT-RL [33], EvoMaster [15], Morest [41], RestCT [61], Schemathesis [28], and MINER [42]. The results demonstrate the superiority of EmRest in operation coverage, code coverage, and bug detection. Specifically, EmRest is able to cover more operations and codes than all baselines in 50% and 44% of APIs, respectively. The effectiveness of EmRest is more evident for APIs with a large number of input parameters and complex constraints, for which 15 operations can be exclusively covered. EmRest is also effective in bug detection, triggering 64% more unique bugs than the baselines across all APIs. Notably, 226 unique bugs can only be detected by EmRest, accounting for 26% of all bugs detected by all approaches.

In summary, the main contributions that this paper makes are as follows:

- We propose EmRest, a novel black-box approach that leverages statistical analysis of error messages to generate valid and exceptional test inputs for effective testing of REST APIs.
- We introduce a method to analyze 400-range error messages to infer invalid combinations of value assignment strategies, enabling a gradual construction and exploration of the valid input space for each operation. We also analyze 500-range error messages to categorize bugs, for which operations that are more vulnerable to parameter validation can be identified and more thoroughly tested.
- We conducted extensive experiments on 16 real-world REST APIs to evaluate EmRest. The experimental results demonstrate the effectiveness of EmRest over SOTA approaches.

```yaml
paths:
  /orders:
    post:
      parameters:
        - in: body
          name: body
          required: true
          schema:
            $ref: '#/definitions/CreateOrder'
      responses:
        201:
          schema:
            type: object
            properties:
              id:
                type: integer
  /customers:
    get:
      responses:
        200:
          schema:
            type: array
            items:
              $ref: '#/definitions/Customer'
  /customers/{id}:
    get:
      parameters:
        - name: id
          in: path
          type: integer

```

```yaml
definitions:
  CreateOrder:
    type: object
    properties:
      id:
        required: true
        type: integer
      type:
        required: true
        type: string
        enum: ['standard', 'express']
      address:
        type: string
        description: Required if 'type' is '
standard'
      priority:
        type: string
        enum: ['low', 'high']
        description: Required if 'type' is 'express
'
  Customer:
    type: object
    properties:
      id:
        type: integer
      regionId:
        type: integer

```

Fig. 1. An OpenAPI Specification Snippet of a Shopping System

2 Background

REST APIs refer to APIs that adhere to the REST architecture style [24]. Such APIs expose internal data as a collection of web resources, each uniquely identified by a Uniform Resource Identifier (URI). Clients interact with these resources by sending HTTP requests, which must specify the resource URI, the HTTP method (e.g., POST, GET, PUT, and DELETE), and input parameter values. Currently, OAS is the most widely used standard for documenting REST APIs, offering a machine-readable format (typically in JSON [5] or YAML [7]) to describe all the necessary information needed to perform API operations.

Fig. 1, for example, gives an OAS snippet of a shopping system. The *paths* field specifies the URIs of three resources, and their allowed HTTP methods. In REST APIs testing, each combination of an HTTP method and a URI constitutes an *operation* (e.g., *POST:/orders*, *GET:/customers*, and *GET:/customers/{id}*). The execution order of operations is critical, as it reflects operation dependencies. According to REST principles, resources are structured hierarchically, as indicated by the forward slashes ('/') in their URIs. The number of segments separated by slashes defines the URI's *depth*. Typically, accessing child resources (with greater depth, e.g., *GET:/customers/{id}*) depends on the availability of parent resources (with lower depth, e.g., *GET:/customers*). Furthermore, HTTP methods guide the execution order for operations on the same resource. Since POST creates resources, it is generally executed first, as subsequent operations (e.g., GET, PUT, DELETE) depend on the created resources.

To perform an operation, the client must also assign appropriate values to its input parameters according to their types, which OAS classifies into six categories: *integer*, *number*, *boolean*, *string*, *array*, and *object*. Among these, *integer*, *number*, *boolean*, and *string* are considered *basic types*, while

array and *object* are *complex types* composed of multiple parameters. For example, in Fig. 1, the operation *POST:/orders* has one object parameter, *CreateOrder*, defined in the *definitions* field. This object contains an integer parameter (*id*) and three string parameters (*type*, *address*, and *priority*). Additionally, OAS specifies the response format for each operation. For example, when executing *GET:/customers*, a 200 status code response contains an array of *Customer* objects, each with two integer parameters, *id* and *regionId*.

Note that constraints typically exist when determining input-parameter values of operations. Such constraints can stem from input parameters within a single operation, including value-specific rules to a specific parameter (e.g., the *type* parameter of *POST:/orders* accepts only '*standard*' or '*express*' as its value), and inter-parameter dependencies [44] (e.g., for *POST:/orders*, the *address* parameter is required if the *type* is '*standard*'). Constraints may also stem from operation dependencies [16], for example, in *POST:/orders*, the *id* parameter must be assigned a valid *id* from responses of *GET:/customers*, which may not necessarily be reflected in the URI hierarchy of resources [16].

When processing an HTTP request, the server returns a response containing an HTTP status code and a response body (typically in JSON format). Status codes in the 200 range indicate success, with response bodies usually providing details about the resource, e.g., *{"id": 10001, "regionId": '20010'}*. In contrast, status codes in the 400 and 500 ranges indicate failures: a 400-range code reflects constraint violations among input parameters, while a 500-range code indicates internal server errors (potential bugs). Response bodies in these cases typically include error-related information (in string format). For instance, an invalid *id* parameter in *POST:/orders* might produce a 400-range error message like *{"message": "Invalid id: Must be a registered customer"}*, whereas a 500-range error message may include an error stack trace. According to REST API best practices (e.g., Microsoft's Azure API guidelines [54]), modern REST APIs should provide informative error messages describing the causes of request failures. Industry standards (e.g., RFC 7807 [50] and 9457 [51]) even force the inclusion of error messages in responses. However, despite the widespread adoption of informative error messages, current testing approaches have not effectively leveraged their diagnostic value. This study aims precisely to exploit these error messages as feedback to enhance REST API testing.

3 The EmRest Approach

In this study, we develop the EmRest approach to generate both valid and exceptional test inputs for REST APIs via analyzing error messages of both 400-range and 500-range status codes. Algorithm 1 gives the main algorithm of EmRest. Given an OAS as the input, EmRest first extracts all available operations O_{all}. Then, EmRest goes through an *Inferring Constraints* phase (lines 2–4) to construct and explore valid input space for every operation in O_{all}, allowing the evaluation of the APIs' behaviors under normal conditions. Here, EmRest especially categorizes O_{all} into two groups: O_{cur} (which contains POST, GET, and PUT operations) and O_{del} (which contains DELETE operations). Operations in O_{cur} are tested earlier than those in O_{del} since DELETE operations remove resources, and these resources are often needed when testing operations in O_{cur}. After this, EmRest further goes through an *Exceptional Testing* phase (line 5), in which invalid inputs are deliberately introduced to test how well each operation in O_{all} handle exceptional scenarios.

The InferringConstraints(O) function (lines 7–16) describes the process to test each of the two operation sets, O_{cur} and O_{del} in the *Inferring Constraints* phase. To account for operation dependencies in REST style (as discussed in Section 2), EmRest sorts all operations in O based on URI hierarchies and CURD semantics (line 8). Here, for O_{cur}, operations with lower URI depths are executed earlier, ensuring that parent resources are created earlier than manipulating their child resources. While for O_{del}, operations with higher URI depths are executed earlier, ensuring that child resources are deleted before deleting their parent resources.

Algorithm 1 The Main Algorithm of EmRest

```
Require: the OpenAPI specification spec
 1: Extract operations O_all from spec
 2: Categorize O_all into O_cur (POST, GET, and PUT operations) and O_del (DELETE operations)
 3: InferringConstraints(O_cur)
 4: InferringConstraints(O_del)
 5: ExceptionalTesting(O_all)
 6:
 7: function InferringConstraints(O)                                          ▷ Section 4
 8:     O ← Sort O by URI hierarchy and HTTP method
 9:     while O is not empty do
10:         op ← Select the head of O
11:         success ← SamplingAndAnalyzingErrorMessages(op)                  ▷ Algorithm 2
12:         if not success and no more than three attempts then
13:             Insert op to O
14:         end if
15:     end while
16: end function
17: function ExceptionalTesting(O)                                           ▷ Section 5
18:     for op in O do
19:         MutateAndCategorizingBugMessages(op)                             ▷ Algorithm 3
20:     end for
21:     while within time budget do
22:         op ← Randomly select an operation from O in proportion to the numbers of previously triggered unique bugs
23:         MutateAndCategorizingBugMessages(op)
24:     end while
25: end function
```

Once operations are sorted, EmRest repeatedly takes one operation *op* from *O* at a time for testing, and repeats until *O* is empty (lines 9–15). This is achieved through the SamplingAndAnalyzingErrorMessages(*op*) function (see Algorithm 2), in which EmRest repeatedly applies combinatorial testing to generate test inputs for *op*. During this process, EmRest manages to statistically analyze error messages of 400-range status code received to identify combinations of value assignments that frequently trigger error messages (i.e., *invalid* combinations for testing *op*), and thus exclude them from future iterations. For EmRest, an operation is considered *successfully tested* if at least one 200-range status code is observed in SamplingAndAnalyzingErrorMessages(*op*), namely, *success* = *true*. If *op* cannot be successfully tested and the number of retries does not exceed three, EmRest will add it back into *O* for further testing (line 13). Here, if *op* is a POST, GET, or PUT operation, it is inserted to the tail of *O*, and will thus be executed only when all preceding operations in *O* have been executed. This is because failures of non-DELETE operations are often caused by the lack of certain resources, and through testing such operations later, the resources required could be created by executing other operations (e.g., in Fig. 1, the *id* parameter of *POST:/orders* is dependent on the response of *GET:/customers*, which is not reflected in the URI hierarchies of these two operations).

Once the valid input spaces of all operations are constructed, EmRest moves to the *Exceptional Testing* phase (line 5). In this phase, the MutatingAndCategoRizingBugMessages(*op*) function (see Algorithm 3) mutates the previously identified valid value assignments of each operation in O_{all}, hoping to generate potentially invalid inputs to test these operations' ability on handling exceptional scenarios. Specifically, EmRest first tests all operations in an initial round, and categorizes error messages of 500-range status code (i.e., bug messages) received to identify unique bugs triggered by each operation (lines 18–20). Since the inability to validate one parameter often indicates potential validation issues in other parameters, possibly leading to additional bugs with distinct

error messages, EmRest assigns each operation a selection probability proportional to the number of unique parameter validation-related bugs previously triggered. In this way, operations prone to parameter validation issues are selected more frequently, enabling thorough testing with a more diverse set of value-assignment mutations.(lines 21–24). The iteration process continues until the predefined time budget is exhausted.

Algorithm 2 SamplingAndAnalyzingErrorMessages(*op*)

Require: the operation to test, *op*
1: Initialize input space $S = \{S_{p_1}, S_{p_2}, \ldots, S_{p_n}\}$ based on specification and historical responses ▷ Section 4.1
2: Initialize the set of invalid combinations of value assignment strategies $C = \emptyset$
3: $\tau \leftarrow 1$, $LC \leftarrow 0$, $EC \leftarrow 0$
4: **while** $LC \leq 10$ **and** $EC \leq 3$ **do**
5: $LC \leftarrow LC + 1$
6: Generate and execute a set of test inputs that cover all combinations of value assignment strategies of any τ parameters based on S and C ▷ Section 4.2
7: Identify invalid combinations by statistically analyzing error messages, and update C accordingly ▷ Section 4.3
8: Increment EC by one if the new error messages identified are identical to those from the previous iterations; otherwise, reset EC to zero
9: Increment τ to two if $EC > 0$; otherwise, reset τ to one
10: **end while**
11: **return** true if at least one 200-range status code is observed

4 Sampling and Analyzing Error Messages

Algorithm 2 gives the implementation of the SamplingAndAnalyzingErrorMessages(*op*) function, which is designed to construct and explore the valid input space for the given operation *op*. Assume that *op* has n input parameters. EmRest first relies on the specification and historical HTTP responses to initialize a list of *value assignment strategies*, S_{p_i}, for each input parameter p_i ($1 \leq i \leq n$), where each strategy in S_{p_i} represents a distinct method for generating concrete values for p_i. The collection of value assignment strategy lists for all n input parameters, $S = \{S_{p_1}, S_{p_2}, \ldots, S_{p_n}\}$, exactly forms the initial *input space* of *op*. By selecting a value assignment strategy $s_{i,j} \in S_{p_i}$ for each input parameter p_i, we can sample an *abstract* test input, $t = \{s_{1,j_1}, s_{2,j_2}, \ldots, s_{n,j_n}\}$, from the input space. Then, by further applying the value assignment strategies specified in t, a *concrete* test input (i.e., an HTTP request) can be constructed and executed.

After the initial input space of *op* is constructed, EmRest then enters a loop (lines 4–10) to test *op*. At each iteration, EmRest first applies combinatorial testing [48] to sample a representative set of test inputs based on the value assignment strategy lists S and the set of constraints C obtained so far (initially, $C = \emptyset$), hoping to trigger diverse behaviors of *op*. Then, after transforming these abstract test inputs into concrete ones and executing them, EmRest collects all error messages of 400-range status codes received, and applies a statistical analysis method to identify invalid combinations of value assignment strategies (i.e., constraints) in S. By keeping adding these invalid combinations into C, EmRest can then avoid generating invalid test inputs in future iterations. The above loop repeats until either the maximum number of iterations LC is reached, or no new error messages are observed in the most recent EC iterations (LC = 10 and EC = 3 in this study).

4.1 Initializing Value Assignment Strategy List

For each operation, EmRest treats each parameter of basic types as a distinct input parameter. While for the parameter of *object* type, EmRest takes a fine-grained strategy that treats each of its leaf parameters (i.e., internal parameters located at the end of the nested structure of the object) as

a distinct input parameter. Then, for each distinct input parameter p_i, EmRest follows the following four categories to determine its list of value assignment strategies, $S_{p_i} = \{s_{i,1}, s_{i,2}, \ldots, s_{i,m}\}$:

- **Null Value Assignment Strategy** (denoted as *NS*()): this strategy assigns a *NULL* value to the input parameter, indicating the parameter will not be included in the test input.
- **Fixed Value Assignment Strategy** (denoted as *FS*(v)): this kind of strategies assigns a single fixed value v that is considered important for testing the parameter.
- **Random Value Assignment Strategy** (denoted as *RS*(r)): this kind of strategies generates a random value satisfying the given requirements r, including the parameter's type, format, and specified constraints (e.g., maximum and minimum for *integer* parameters).
- **Resource-Based Value Assignment Strategy** (denoted as *RBS*(*op*, *field*)): this kind of strategies retrieves a runtime value from the *field* in the previously obtained HTTP responses of operation *op*, and assigns it to the input parameter.

For each p_i, if p_i is optional, a *NS*() strategy is added to S_{p_i}. Next, if p_i has an *enum* field, EmRest adds one *FS*() strategy for each of the enumerated values specified into S_{p_i}, and completes the initialization process (as these explicitly define all valid assignments for p_i). If no *enum* field is present in p_i, EmRest then extracts all fixed values from p_i's fields in the specification, and employs each value to define a *FS*() strategy. Specifically, each individual value listed in *example* and *default* fields is extracted. Each character sequence enclosed in single or double quotes within the *description* field (as they are likely example values [32]) is also extracted. For parameters of *string*, *number*, and *integer* types, EmRest further defines *FS*() strategies for boundary values: a zero-length string for *string* typed parameters, and the values 0, 1, and -1 for *integer* and *number* typed parameters.

Next, EmRest adds one *RS*() into S_{p_i}. Note that it can be difficult to generate valid values for parameters of *string* type, as valid strings usually need to conform to specific formats. So, if a *string* typed parameter has no format specified, EmRest further follows the built-in string formats of OAS [1] to add three *RS*() strategies into S_{p_i}: one that generates binary strings, another that generates base64-encoded byte strings, and a third that generates password strings (combinations of uppercase, lowercase, digits, and punctuation).

Then, EmRest adds a series of *RBS*() strategies into S_{p_i}, in order to take operation dependencies into consideration (this kind of strategies will only be added when there are testing results). For this, EmRest maintains a pool of 200-range HTTP responses for each operation, each of which is associated with a response schema definition. Then, for each response pool, EmRest employs the *token_set_ratio* method of the *FuzzyWuzzy* library [3] to measure the token set similarity score $sim \in [0, 100]$ between the name of p_i and each response schema field. If *sim* exceeds 60 (which is a relatively low threshold, as we would like to include as many relevant fields as possible), EmRest collects all values under this field name from the response pool to construct a *RBS* strategy. This strategy will randomly select one of these collected values for assigning p_i.

For example, in the *POST:/orders* operation from Fig. 1, it has one *object* parameter, *CreateOder*, which expands into four distinct input parameters: *id*, *type*, *address*, and *priority*. For the input parameter *type*, since it has an *enum* field, we have $S_{type} = \{FS(\text{`}standard\text{'}), FS(\text{`}express\text{'})\}$, which contains two fixed value assignment strategies only (as specified by the two values in the *enum* field). For the input parameter *address*, since it is optional, a null value assignment strategy, *NS*(), is added. Then, there are two character sequences '*type*' and '*standard*' enclosed in single quotes within its *description* field, two fixed value assignment strategies, *FS*('*type*') and *FS*('*standard*'), are added. Furthermore, as a *string* parameter, another fixed value assignment strategy *FS*('') is added (generating a zero-length string). Then, since no format is specified for this *string* parameter, four random value assignment strategies are initialized: *RS*(*string*), *RS*(*binary*), *RS*(*byte*), and

RS(*password*). Now, suppose that the *GET:/customers* operation returns a response containing a *Customer* object with two output parameters: *id* and *regionId*. If *address* is determined to be sufficiently similar to these output parameter names, two resource-based value assignment strategies are added: *RBS*('*GET:/customers*', '*id*') and *RBS*('*GET:/customers*', '*regionId*'). Finally, we have $S_{address}$ = {*NS()*, *FS*('*type*'), *FS*('*standard*'), *FS*(''), *RS*(*string*), *RS*(*binary*), *RS*(*byte*), *RS*(*password*), *RBS*('*GET:/customers*', '*id*'), *RBS*('*GET:/customers*', '*regionId*')} as its value assignment strategy list.

4.2 Generating and Executing Test Inputs

At each iteration of Algorithm 2, EmRest applies combinatorial testing (specifically, the PICT [9] tool) to generate test inputs based on the value assignment strategy lists of the operation, *S*, and the collection of invalid combinations of value assignment strategies, *C*. Combinatorial testing is a popular test input space sampling technique [21, 49, 56, 60, 61], which uses a τ-way *covering array* as the test suite to systematically cover all possible combinations of any τ parameter values. For each operation, EmRest first generates a 1-way covering array as the set of abstract test inputs, and then constructs one concrete test input for each abstract test input. This ensures that every value assignment strategy in *S* will be tested at least once, and accordingly, helps to collect error messages related to the violation of constraints on a single input parameter. Next, if the error message observed in this iteration is the same as that from the most recent previous round (*EC* > 0 in Algorithm 2), EmRest increases the value of τ to two, so the combinations between value assignment strategies of any two input parameters can be further tested. This helps not only detect interaction-related bugs in REST APIs, but also collect error messages related to violations of inter-parameter constraints.

Fig.2 uses an example to illustrate the workflow of EmRest for testing the *POST:/orders* operation in Fig. 1 (with $\tau = 2$). For simplicity, we consider a reduced list of value assignment strategies for each of the four input parameters, *id*, *type*, *address*, and *priority* (with $|S_{id}| = |S_{type}| = |S_{address}| = |S_{priority}| = 2$), as shown in Fig.2 (a). Fig 2 (b) gives a 2-way covering array generated (without constraints), where each row indicates an abstract test input. Note that these six test inputs cover all possible combinations of any two parameters (e.g., for *id* and *type*, all $2 \times 2 = 4$ strategy combinations appear at least once in the array). Then, by applying value assignment strategies specified in each abstract test input, one concrete test input is generated, as shown in Fig. 2 (c). After executing these concrete test inputs, the HTTP responses, including status codes and error messages, as shown in Fig 2 (d), are collected for further analysis.

Note that prior studies [36, 48] indicate that most software failures result from interactions among only a few parameters, with 2-way parameter combinations accounting for the majority. Moreover, a large-scale empirical analysis of REST APIs [44] showed that inter-parameter constraints involving more than two parameters occur in merely 14% of cases. Thus, 2-way combinations already cover most real-world constraint scenarios. Although complex API operations can involve many input parameters and *object*-type parameters, significantly enlarging the input space, previous studies [38, 39] demonstrated that small-sized 2-way covering arrays could still be efficiently generated even for systems with hundreds of parameters. Considering these findings, EmRest chooses to use up to 2-way covering arrays, balancing effectiveness and scalability.

4.3 Determining Invalid Combinations of Value Assignment Strategies

Once the concrete test inputs are executed and the testing results are obtained, EmRest applies statistical analysis to infer constraints from the received 400-range error messages. For EmRest, a *constraint* in the input space *S* is expressed as a combination of *k* value assignment strategies (selected from *k* different strategy lists), denoted as $\{p_{i_1} = s_{i_1,j_1}, p_{i_2} = s_{i_2,j_2}, \ldots, p_{i_k} = s_{i_k,j_k}\}$ where $s_{i,j}$ is

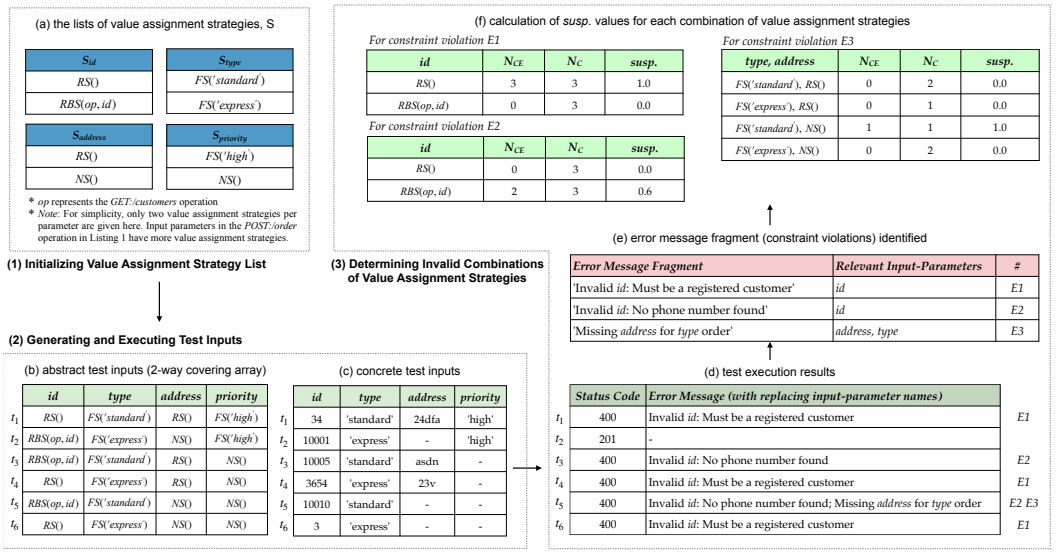


Fig. 2. The workflow of *Sampling and Analyzing Error Messages* in EmRest.

the j-th strategy selected from S_{p_i}. Such a combination is also referred to as an *invalid combination of value assignment strategies*, as its application in generating concrete values will produce 400-range status codes. For example, for the *POST:/orders* operation in Fig. 1, {*type* = *FS*('*standard*'), *address* = *NS()* } is an invalid combination, as the OAS specifies that *address* is required if *type* is '*standard*'.

4.3.1 Identifying Error Message Fragments. Given a 400-range HTTP response body and its corresponding concrete test input, EmRest extracts each field value from the response body (except for timestamp, status code, path, and URI that are unlikely to contain error related information) as a single error message (a string of text). Then, EmRest relies on the concrete input parameter values found in each error message to associate it with its relevant input parameters. On this basis, EmRest further replaces each concrete value with its corresponding parameter name to distinguish distinct errors. For example, in the *POST:/orders* operation from Fig. 1, if *id* is assigned an invalid value, such as 34, the server may return the error message: "Invalid 34: Must be a registered customer". EmRest replaces 34 with the corresponding parameter name, transforming the message into: "Invalid *id*: Must be a registered customer." As such, error messages describing the same error can be categorized (e.g., assigning either 34 or 35 to *id* triggers the same error, even though the original error messages received are different). For simplicity, the error messages displayed in Fig. 2 (d) are those after replacing concrete input parameter values.

Note that an error message might describe multiple constraint violations when the test input violates multiple constraints. So, to track each distinct violation, EmRest further identifies *error message fragments*, where each fragment is a snippet text of the error message that describes a single constraint violation and does not contain any other error message fragment. Specifically, given a set of previously identified error message fragments (initially empty when EmRest starts) and a set of newly observed error messages, EmRest first treats each new error message as a new error message fragment. Then, EmRest iteratively compares every pair of fragments to determine whether one fragment contains another, and keeps splitting and replacing the longer fragment with

the shorter one. This process repeats until no two fragments contain each other. At this moment, all fragments remaining are error message fragments.

For example, in Fig. 2 (d), EmRest initially collects a set of three error messages: "Invalid *id*: Must be a registered customer." (from t_1, t_4, and t_6), "Invalid *id*: No phone number found." (from t_3), and "Invalid *id*: No phone number found; Missing *address* for *type* order" (from t_5). Since the third message contains the second, so EmRest splits the third into two messages: "Invalid *id*: No phone number found" and "Missing *address* for *type* order". After removing the original third message and add the two newly identified messages, EmRest confirms the final three messages as the three error message fragments identified (because they do not contain each other). Fig. 2 (e) shows these three identified error message fragments, E_1, E_2, and E_3, and their relevant input parameters.

4.3.2 Determining Invalid Combinations. After identifying the set of error message fragments (i.e., distinct constraint violations), EmRest determines the invalid combinations of value assignment strategies that contribute to each constraint violation. To achieve this, a straightforward approach is to directly mark the combinations used in the test inputs that trigger any error message fragment as invalid. However, this tends to overestimate constraints in the input space, as some value assignment strategies do not always produce values that violate constraints. For example, in Fig. 2 (b) and (d), the *RBS*(*op*, *id*) strategy for parameter *id* is only invalid when the retrieved customer lacks a phone number. Simply discarding this strategy may overly restrict the input space, and if it's the only strategy producing valid values, *id* may fail to obtain a valid value in further testing.

To accurately infer constraints, EmRest adopts a statistical approach to assess the likelihood of each combination of value assignment strategies triggering each error message fragment. The core idea behind this step is that if the use of a certain combination of value assignment strategies causes test inputs triggering the same error message more frequently, then this combination is more likely to be invalid. For each error message fragment E, EmRest first identifies its relevant input parameters and collects all combinations of value assignment strategies used for these parameters across the set of test inputs. Then, for each strategy combination c, EmRest calculates the number of test inputs where c is used, denoted as N_C, and the number of times the use of c leads to E, denoted as N_{CE} (all previously executed test inputs are also considered in this step to enable a more accurate estimation) Finally, the likelihood of strategy combination c triggering error message fragment E is computed as $susp. = N_{CE}/N_C$. If the suspicious value *susp.* exceeds a predefined threshold, then c is identified as an invalid combination of value assignment strategies.

For example, Fig. 2 (f) illustrates the calculation of *susp.* values for the three error message fragments. For E_1 and E_2, they are relevant to parameter *id* only, which corresponds to two strategy combinations[1] for calculation. While for E_3, it is relevant to two parameters, and there are thus four strategy combinations for calculation (note that these four combinations ensure to appear in Fig. 2 (b), because a 2-way covering array is used). Let us then take E_3 as the example. For the first strategy combination, {*address* = *RS*(), *type* = *FS('standard')*}, we have $N_C = 2$ because this combination is used twice in Fig. 2 (b) (t_1 and t_3); and $N_{CE} = 0$ because none of these uses results in error message fragment E_3 (as t_1 and t_3 trigger E_1 and E_2, respectively). As such, the *susp.* value of this combination is as $0/2 = 0$. Similarly, we can calculate the *susp.* value of {*id* = *RS*()} for E_1 as $3/3 = 1$ (all three uses of this strategy, in t_1, t_4, and t_6, result in E_1). At this moment, if the threshold is set to 0.7, then {*id* = *RS*()} will be identified as an invalid combination (i.e., a constraint).

Note that to ensure the accuracy of identifying invalid combinations, the threshold should be close to 1.0, maximizing confidence that the identified combinations are truly invalid. However, in practice, some errors may mask others, meaning that even if a test input contains two invalid

[1]For consistency, in this study, we refer to each single value assignment strategy as also a strategy *combination* (i.e., 1-way combination [48]).

combinations, the received error message may include only one error message fragment. In such cases, a relatively low threshold can help identify invalid combinations related to the masked error, as N_{CE} tends to be small. In this study, we set the threshold to 0.7 to balance accuracy and efficiency in identifying invalid strategy combinations.

Algorithm 3 MutateAndCategorizingBugMessages(*op*)

Require: the operation to mutate *op*
1: Let S and C be the value assignment strategy lists and constraints obtained at the end of *Inferring Constraints* phase
2: $LC \leftarrow 0$, $BC \leftarrow 0$
3: **while** $LC \leq 10$ **and** $BC \leq 3$ **and** within time budget **do**
4: $LC \leftarrow LC + 1$
5: Generate a set of abstract test inputs T_a based on S and C
6: **for** each abstract test input t **in** T_a **do**
7: **if** the content type of t is not null **and** random.uniform(0, 1) < 0.5 **then**
8: Apply *Content Type Mutation* operator to the content type of t
9: **end if**
10: **for** each input parameter p **in** t **do**
11: **if** random.uniform(0, 1) < 0.5 **then**
12: Apply *Input Type Mutation* operator to replace value assignment strategy of p to an exceptional strategy
13: **end if**
14: **end for**
15: Construct and execute a concrete test input of t
16: **end for**
17: Collect and categorize bug messages
18: Increment BC by one if the newly identified error messages are identical to those from the previous iteration; otherwise, set BC to zero
19: **end while**

5 Mutating and Categorizing Bug Messages

Algorithm 3 outlines the implementation of the MutatingAndCategorizingBugMessages(*op*) function, which aims to generate invalid test inputs to uncover as many parameter validation-related bugs as possible (i.e., exceptional testing [33, 59]). To this end, EmRest first reuses all lists of value assignment strategies S and the set of constraints C obtained at the end of the *Inferring Constraints* phase as the valid input space of *op*. Then, similar to the SamplingAndAnalyzingErrorMessages(*op*) function, EmRest repeatedly applies combinatorial testing to sample a series of valid abstract test inputs, but here, these abstract test inputs will be further mutated before constructing concrete test inputs (line 5–16).

Specifically, for each constraints-satisfying abstract test input t, EmRest applies two kinds of mutation operators to generate exceptional test input. First, if t has a request body, EmRest applies the *Content Type Mutation* operator (with probability 50%) to randomly change its content type into a different choice, as suggested in [33] (lines 7–9). Second, for each input parameter p in t, EmRest applies the *Input Type Mutation* operator (with possibility 50%) to randomly assign p an exceptional strategy that generates random values of an incompatible type (e.g., a *string* value for an *integer* parameter). To this end, EmRest maintains a set of random value assignment strategies as candidate exceptional strategies, each corresponding to a unique parameter type defined by OAS. When the *Input Type Mutation* operator is applied to parameter p, EmRest randomly selects one strategy from these candidates that does not match p's type as its exceptional strategy. To encourage diverse and thorough testing of API input validation, EmRest initially assigns each exceptional strategy a weight of 1.0 per parameter. Once an exceptional strategy is selected, its weight is reduced by a factor of 0.9, thus increasing the chance of selecting other strategies to trigger

different parameter validation-related bugs. During the execution of the generated concrete test inputs, EmRest continuously collects error messages of 500-range status code (i.e., bug messages), and applies the same fragment identification process (see Section 4.3.1) to identify distinct bug message fragments. This sampling and mutation process repeats until either a predefined iteration limit LC is reached, or no new bug message fragments appear in the most recent BC iterations (LC = 10 and BC = 3 in this study).

6 Experiment

This section presents the experiment conducted to assess the effectiveness and efficiency of EmRest. In particular, we focus on the following three research questions:

RQ1: How effective is EmRest in achieving operation and code coverage for REST APIs?
RQ2: How effective is EmRest in detecting bugs in REST APIs?
RQ3: How does the operation selection strategies impact the effectiveness of EmRest?

6.1 Experiment Setup

Table 1. The Subject APIs

REST APIs (*abbr*)	Description	# Operations	# Input Parameters
Features Service (*FeatSrv*)	Manages product features for product-related functionality.	18	1.9
Genome Nexus (*GenomeNex*)	Provides tools for annotating and interpreting genetic variants in cancer.	23	2.2
Language Tool (*LangTool*)	Open-source proofreading tool that supports multiple languages.	2	5.5
Market Service (*MktSrv*)	A project focused on Spring and web technologies for backend development.	13	3.6
Person Controller (*PersCtrl*)	Java project using MongoDB for easy data management.	12	5.0
Project Tracking System (*ProjTrack*)	Software for managing and tracking project progress.	60	5.2
REST Countries (*RestCtry*)	Get information about countries.	22	1.5
User Management Microservice (*UserMgmt*)	Manages user data for online systems.	22	2.9
NCS (*NCS*)	An artificial software for mathematical and statistical computations.	6	2.3
SCS (*SCS*)	Tool for calculations and text data processing.	11	2.4
GitLab Branch (*GLBranch*)	Manages branches in Git repositories.	9	10.7
GitLab Commit (*GLCommit*)	Tracks commits in Git repositories.	15	10.0
GitLab Groups (*GLGroups*)	Manages teams within GitLab.	17	8.9
GitLab Issues (*GLIssues*)	Tracks project issues and bug reports.	27	10.1
GitLab Project (*GLProject*)	Manages GitLab projects throughout their lifecycle.	31	10.0
GitLab Repository (*GLRepo*)	Manages data and versions in Git repositories.	10	10.9

6.1.1 Subject APIs. In this study, we selected 16 representative real-world REST APIs as experimental subjects. Table 1 lists their names, descriptions, number of operations, and average number of input parameters per operation. The first ten APIs were obtained from a SOTA REST API testing approach, ARAT-RL [33]. According to [33], these APIs were chosen from a previous study [35], which initially considered 20 diverse APIs. Some APIs were excluded in [33] due to requiring domain-specific knowledge or relying on outdated external dependencies. Additionally, APIs with authentication issues or restrictive rate limits were also excluded, as these factors could impact automated test generation evaluations. Ultimately, ten APIs remained in [33]. To ensure a fair comparison, we directly reused the exact specifications of these ten APIs in our experiments.

In addition, to evaluate more complex scenarios, we included six APIs from GitLab, an enterprise-level Git repository management system. GitLab APIs provide operations on core resources such as branches, projects, groups, issues, commits, and repositories. These APIs are frequently used in recent REST API testing studies [16, 40, 42, 61]. As shown in Table 1, GitLab APIs generally involve more input parameters per operation compared to the first ten APIs, making them suitable for evaluating complex constraint inference and input-space exploration. We obtained the specifications of GitLab APIs from a previous study [61]. Since those specifications were manually created, we further validated them against GitLab's official documentation (v14.4.2, a more recent version), and made corrections when necessary. Our revised specifications are included in the supplementary materials.

6.1.2 Evaluation Baselines. In this study, we selected six SOTA REST API testing approaches as baselines for comparison: ARAT-RL [33], EvoMaster (black-box version)[15], Morest[41], RestCT [61], Schemathesis [28], and MINER [42]. Specifically, we first considered an initial set of ten approaches from a recent empirical study [35], selecting the top three black-box approaches in terms of code coverage and bug detection: EvoMaster [15], RESTler [16], and Schemathesis [29]. To provide a more comprehensive evaluation and account for recent advancements, we further included four newly proposed SOTA approaches: ARAT-RL [33], MINER [42], Morest [41], and RestCT [61]. Since MINER [42] extends RESTler by integrating an attention model into the testing process, it inherently evaluates RESTler's effectiveness. Additionally, previous studies [33, 42] consistently showed MINER and ARAT-RL outperforming RESTler. Therefore, we excluded RESTler to avoid redundancy.

To further investigate the contribution of different components in EmRest, we compared it with three variants: EmRest$_{Infer}$, EmRest$_{Random}$, and EmRest$_{NoRetry}$. Specifically, EmRest$_{Infer}$ excludes the *Exceptional Testing* phase, evaluating the respective impacts of the *Inferring Constraints* and *Exceptional Testing* phases on performance (*RQ1* and *RQ2*). EmRest$_{Random}$ modifies the operation selection strategy in the *Inferring Constraints* phase: rather than categorizing operations into O_{cur} and O_{del} and selecting based on URI hierarchy and HTTP methods, it randomly selects operations from O_{all} (line 10, Algorithm 1). Finally, EmRest$_{NoRetry}$ restricts each operation to be tested only once during the *Inferring Constraints* phase. If an operation does not yield a 200-range status code, it will not be re-inserted into O for additional testing (line 13, Algorithm 1). The variants EmRest$_{Random}$ and EmRest$_{NoRetry}$ aim to explore how operation selection strategies influence overall performance (*RQ3*).

6.1.3 Experimental Procedure. We executed each testing approach and each variant of EmRest for one hour per subject API. This duration aligns with recommendations from recent studies [33, 35], indicating REST API testing approaches typically plateau in performance within one hour, with code coverage after one hour being comparable to that achieved after 24 hours

To account for inherent randomness, we repeated each experiment 30 times per API. Before each run, we deleted and redeployed the API to ensure a clean, consistent environment. All experiments were conducted on a server with an Intel(R) Xeon(R) Gold 5118 CPU @ 2.30GHz (48 cores), 128GB RAM, running CentOS 7. For data collection, we used Mitmproxy [6] to capture all HTTP requests and responses during testing. To measure code coverage, we employed JaCoCo [4] for the first ten Java-based APIs. For the GitLab APIs, we utilized a Docker image with built-in coverage support provided by [40].

Based on the collected data, we measured and compared the *operation coverage*, *code coverage*, and *bug detection* of each testing approach. An operation is considered *covered* if it returns at least one 200-range HTTP response [45]. Code coverage is measured as the percentage of lines executed by the generated test inputs. For bug detection, we count the number of unique bugs identified by each approach. Following prior work [33], we classify unique bugs based on response messages received for each operation. Specifically, we remove unrelated information, such as timestamps, from the responses and define a *unique bug* as a combination of an operation and its distinct response message instance.

6.2 Experiment Results

6.2.1 RQ1 (Operation and Code Coverage). Fig. 3 and Fig. 4 show the average number of operations covered and line coverage achieved by different testing approaches across 30 runs, respectively. Each subplot corresponds to a subject API, with error bars indicating standard deviations (shorter bars imply more stable performance).

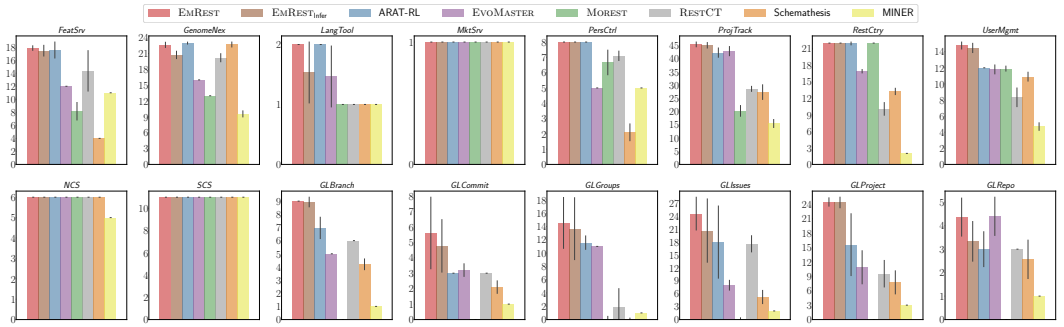


Fig. 3. Number of Operations Covered By Different Testing Approaches

From Fig. 3, EmRest generally achieves the best operation coverage, followed by ARAT-RL, EvoMaster, RestCT, and Schemathesis. Specifically, EmRest covers more operations than all six baselines in eight out of 16 APIs and matches the best baseline in six others. Even in the two APIs (*GenomeNex* and *GLRepo*) where EmRest performs slightly worse, the coverage difference is within 0.33 operations. The advantage of EmRest is particularly evident for GitLab APIs. For instance, in *GLProject*, it covers up to 8.87 more operations than ARAT-RL. Moreover, approaches such as Morest and MINER cover only up to 22.9% of operations achieved by EmRest on these APIs, despite their relatively strong performance on the first ten APIs. This indicates potential shortcomings in generating valid test inputs under complex constraints.

Overall, across all APIs, EmRest covers 30.3 more operations in total than the best baseline, ARAT-RL. The set of operations covered by EmRest generally includes those covered by other baselines (only four operations are uniquely covered by other baselines). EmRest exhibits particularly stable performance on the first ten APIs, with an average standard deviation of only 0.2. For the more complex GitLab APIs, this value increases to 1.9, indicating higher randomness (though still lower than ARAT-RL's 2.9). Additionally, EmRest and EmRest$_{Infer}$ (without the *Exceptional Testing* phase) achieve nearly identical operation coverage on most APIs. This is because the goal of *Exceptional Testing* is to evaluate APIs' parameter-validation capabilities by generating inputs intended to trigger 500-range errors rather than successful 200-range responses, thus having minimal impact on operation coverage.

Moreover, among all operations covered across APIs, 15 operations (one in *ProjTrack*, one in *GLGroups*, three in *UserMgmt*, and ten in *GLCommit*) can only be covered by EmRest. Analyzing these operations reveals that they typically involve multiple interleaving constraints, usually absent from the OAS. For instance, in the *POST:/users* operation of *UserMgmt*, there are 21 input parameters with complex constraints: four parameters are mandatory; *gender* can only be *male* or *female*; *username* and *email* must be unique; and *password* must follow a specific format. Existing approaches may identify valid values for individual parameters (e.g., extracting valid *email* from responses or using specific strategies to generate a valid *password*). However, as the number and complexity of input parameters increases, these methods often fail to satisfy multiple constraints simultaneously. Although status-code-based feedback can guide parameter adjustments, modifying one parameter to meet a constraint often violates another. In contrast, EmRest employs statistical analysis to infer invalid combinations from error messages. It does not require explicit constraint descriptions in the OAS and can identify multiple invalid combinations based on a single constraint violation (see Section 4.3). Therefore, EmRest performs better on APIs involving complex constraints.

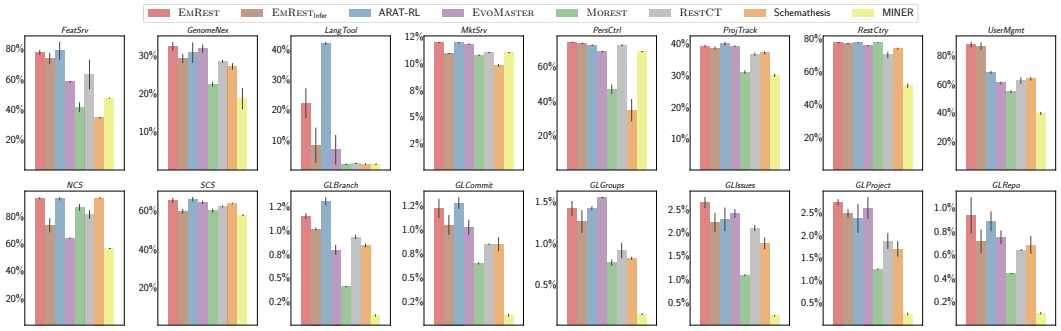


Fig. 4. Line Coverage Achieved By Different Testing Approaches

Regarding code coverage (Fig.4), EmRest achieves the highest line coverage in seven out of 16 APIs, with an average improvement of 10.5%. However, for eight APIs, EmRest slightly underperforms compared to baselines, particularly against ARAT-RL on *LangTool* (where both approaches achieve the same operation coverage, see Fig.3). For the *POST:/check* operation in *LangTool*, assigning different valid values to the *language* parameter typically results in distinct execution paths. Certain values (e.g., '*ga*') lead to more extensive code execution. Here, the value reuse strategy in ARAT-RL frequently reuses such values, whereas EmRest assigns other valid alternatives. Nevertheless, EmRest's stronger ability to cover more operations remains beneficial for code coverage. For example, in *UserMgmt* (where EmRest covers 2.8 more operations), EmRest achieves up to 28.7% higher line coverage compared to ARAT-RL.

Answer to RQ1: EmRest is the most effective approach for operation coverage, outperforming all six baselines in eight of the 16 APIs, with up to 8.87 additional operations covered in the more complex GitLab API (*GLProject*). Additionally, across all APIs, 15 operations can only be covered by EmRest.

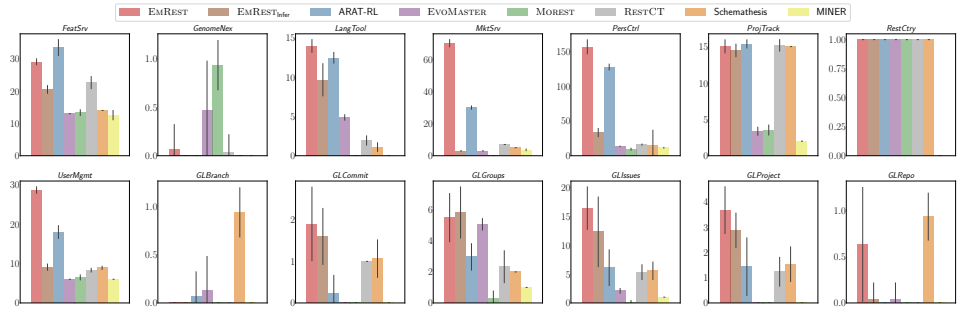


Fig. 5. Number of Unique Bugs Detected by Different Approaches (no bugs were detected in *NCS* and *SCS*)

6.2.2 RQ2 (Bugs Detection). Fig.5 shows the average number of unique bugs detected by each approach across 30 runs. Similar to Fig.3 and Fig. 4, each subplot corresponds to one subject API, with error bars indicating standard deviations. Two APIs (*SCS* and *NCS*) are excluded because no bugs were detected, as they were artificially developed for evaluating code coverage in unit tests. According to Fig. 5, EmRest detects more unique bugs than all six baselines in eight out of 16

APIs and achieves equal or better results in 11 APIs. The most significant improvement occurs in *MktSrv*, where EmRest detects, on average, 40 more unique bugs than the second-best approach, ARAT-RL (a 134% improvement). Generally, ARAT-RL is the strongest baseline identified, followed by RestCT, Schemathesis, and Morest. Although operation coverage does not strongly correlate with bug detection, standard deviations follow a similar pattern, exhibiting stable performance for the first ten APIs and relatively higher variability for GitLab APIs.

In addition, Fig. 5 shows that EmRest detects notably more unique bugs than EmRest$_{Infer}$, especially for the first ten APIs. This is primarily due to the critical role of *Exceptional Testing* in exposing parameter validation-related bugs, which dominate bug detection in these APIs (originally developed as benchmarks without strict parameter validation). However, in more mature enterprise APIs like GitLab, parameter validation bugs are rare, and most bugs stem from business logic errors. Since *Exceptional Testing* contributes less in these cases, EmRest and EmRest$_{Infer}$ achieve similar bug detection results. Nevertheless, EmRest still outperforms all baselines on GitLab APIs due to its use of combinatorial testing, which systematically examines inter-parameter interactions (e.g., a bug in *GET:/groups* from *GLGroups* appears only when *per_page* > 0, *statistics* = *true*, and *min_access_level* is used).

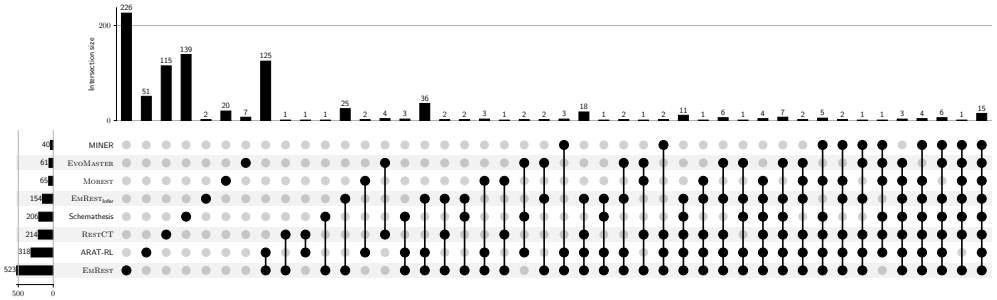


Fig. 6. Overlap of All Detected Unique Bugs Across 30 Runs

Fig. 6 visualizes the overlap of unique bugs detected by all approaches across 30 runs. The leftmost horizontal bar shows the total unique bugs detected by each approach (e.g., EmRest detects 523 unique bugs). The upper vertical bars represent bugs exclusively detected by specific sets of approaches, marked by black dots (e.g., the eighth column shows 125 bugs detected only by EmRest and ARAT-RL).According to Fig. 6, EmRest identifies 523 unique bugs, accounting for 60% of all 867 bugs found. Notably, 226 bugs can only be detected by EmRest, whereas the second and third best approaches, Schemathesis and RestCT, exclusively detect 139 and 115 bugs, respectively. Although ARAT-RL detects more bugs overall compared to Schemathesis and RestCT, most of these bugs are also identified by other approaches.

Further analysis of bugs exclusively detected by EmRest shows that they are typically parameter validation related and highly concentrated in seven operations from *PersCtrl* and *MktSrv* APIs. These operations involve many input parameters, averaging 13 compared to 4.1 parameters for operations where EmRest detects no bugs. In such cases, randomly selecting operations for exceptional testing (as done by baselines) is usually ineffective. In contrast, EmRest prioritizes operations prone to parameter validation issues (Algorithm 1) and selects mutation operators that are less frequently applied to each parameter (Algorithm 3), ensuring comprehensive validation logic examination. These findings further demonstrate the effectiveness of the *Exceptional Testing* phase in detecting bugs.

Answer to RQ2: EmRest detects more unique bugs than all six baselines in eight of the 16 APIs (up to 134% improvement). Overall, EmRest detects 60% of all unique bugs found in the experiments, with 26% detected exclusively by EmRest.

Table 2. Covered Operations, Line Coverage, and Detected Unique Bugs for EmRest, EmRest$_{Random}$, and EmRest$_{NoRetry}$ Across 16 Subject APIs Over 30 Runs

	Operation Coverage			Line Coverage			Unique Bugs		
	EmRest	EmRest$_{Random}$	EmRest$_{NoRetry}$	EmRest	EmRest$_{Random}$	EmRest$_{NoRetry}$	EmRest	EmRest$_{Random}$	EmRest$_{NoRetry}$
FeatSrv	17.9 ± 0.3	17.9 ± 0.3	18.0 ± 0.0	77.3% ± 1.4	77.6% ± 1.4	77.6% ± 0.8	29.1 ± 0.9	29.6 ± 1.2	29.2 ± 1.2
GenomeNex	22.6 ± 0.5	22.7 ± 0.6	22.5 ± 0.6	32.2% ± 1.0	30.5% ± 1.6	31.0% ± 1.3	0.1 ± 0.3	0.0 ± 0.2	0.0 ± 0.2
LangTool	2.0 ± 0.0	2.0 ± 0.0	2.0 ± 0.0	22.1% ± 4.7	23.0% ± 4.8	23.6% ± 4.5	14.0 ± 0.8	13.6 ± 1.1	14.5 ± 1.0
MktSrv	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	11.9% ± 0.0	11.9% ± 0.0	12.1% ± 0.9	69.9 ± 2.2	70.7 ± 2.2	70.3 ± 2.2
PersCtrl	8.0 ± 0.0	7.5 ± 0.7	8.0 ± 0.0	74.0% ± 0.0	73.1% ± 1.3	74.0% ± 0.0	156.6 ± 10.0	166.8 ± 17.7	173.9 ± 9.9
ProjTrack	45.4 ± 0.9	42.9 ± 1.7	43.6 ± 1.2	39.3% ± 0.3	39.0% ± 0.4	39.0% ± 0.4	15.0 ± 0.9	11.3 ± 1.7	15.1 ± 1.0
RestCtry	22.0 ± 0.0	21.9 ± 0.6	22.0 ± 0.0	77.5% ± 0.0	77.5% ± 0.2	77.5% ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
UserMgmt	14.8 ± 0.4	14.8 ± 0.4	13.6 ± 0.5	87.4% ± 1.6	87.6% ± 1.3	80.5% ± 3.1	28.6 ± 0.8	28.5 ± 1.1	28.1 ± 0.7
NCS	6.0 ± 0.0	6.0 ± 0.0	6.0 ± 0.0	92.9% ± 0.6	92.8% ± 0.1	92.8% ± 0.2	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
SCS	11.0 ± 0.0	11.0 ± 0.0	11.0 ± 0.0	65.3% ± 1.1	65.7% ± 0.9	65.2% ± 1.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
GLBranch	9.0 ± 0.0	7.6 ± 1.4	4.7 ± 2.5	1.1% ± 0.0	1.1% ± 0.1	0.9% ± 0.2	0.0 ± 0.0	0.1 ± 0.4	0.0 ± 0.0
GLCommit	5.6 ± 2.3	5.3 ± 2.0	2.6 ± 1.7	1.2% ± 0.1	1.2% ± 0.1	0.9% ± 0.2	1.9 ± 0.9	2.0 ± 0.8	1.0 ± 0.9
GLGroups	14.5 ± 3.8	5.1 ± 3.6	4.9 ± 5.6	1.4% ± 0.1	1.2% ± 0.1	1.2% ± 0.1	5.5 ± 1.5	3.2 ± 1.4	3.4 ± 2.1
GLIssues	24.6 ± 3.8	19.6 ± 7.4	8.8 ± 8.5	2.6% ± 0.1	2.5% ± 0.2	2.2% ± 0.3	16.4 ± 3.7	12.4 ± 6.0	5.9 ± 5.9
GLProject	24.4 ± 0.9	18.8 ± 3.7	13.7 ± 10.5	2.7% ± 0.1	2.5% ± 0.2	2.2% ± 0.5	3.7 ± 0.9	2.6 ± 1.2	1.8 ± 1.9
GLRepo	4.4 ± 0.8	3.9 ± 1.1	1.8 ± 1.3	0.9% ± 0.2	0.8% ± 0.1	0.6% ± 0.1	0.6 ± 0.6	0.6 ± 0.6	0.2 ± 0.4

6.2.3 RQ3 (Impact of Operation Selection Strategies). Table 2 presents operation coverage, line coverage, and the number of unique bugs detected by EmRest and its two variants (EmRest$_{Random}$, selecting operations randomly, and EmRest$_{NoRetry}$, testing each operation only once) across 30 runs. Results are formatted as *average* ± *standard deviation*. From Table 2, EmRest$_{Random}$ and EmRest$_{NoRetry}$ achieve similar operation and line coverage to EmRest for the first ten APIs (except for *ProjTrack*), but EmRest greatly outperforms both variants on the GitLab APIs. By analyzing APIs where EmRest$_{Random}$ and EmRest$_{NoRetry}$ perform worse, we find they typically involve operation dependencies not reflected by the URI hierarchy. For instance, in *ProjTrack*, four operations with identical URI lengths must execute sequentially (*POST:/authenticate*, *POST:/employees*, *POST:/projects*, *POST:/assignment*). Here, EmRest$_{NoRetry}$ may miss testing operations whose dependencies were initially unsatisfied, while EmRest$_{Random}$ can only discover correct operation orders by chance, resulting in lower performance compared to EmRest.

In terms of bug detection, EmRest performs similarly to the two variants for the first ten APIs. In some cases, such as *PersCtrl*, EmRest$_{NoRetry}$ detects 17.3 more unique bugs than EmRest. This occurs because, without retries, EmRest$_{NoRetry}$ completes the *Inferring Constraints* phase faster, allowing more time for the *Exceptional Testing* phase and thus uncovering more parameter validation-related bugs within the same test budget. However, for GitLab APIs, both EmRest$_{Random}$ and EmRest$_{NoRetry}$ detect significantly fewer bugs than EmRest, since GitLab bugs typically stem from complex parameter interactions (as discussed in RQ2), making the effective capturing of operation dependencies essential. This result highlights the effectiveness of EmRest's operation selection strategies.

Answer to RQ3: EmRest's operation selection strategy effectively enhances test coverage and bug detection, particularly when dealing with complex operation dependencies.

7 Threats To Validity

Regarding internal threats to validity, the performance of EmRest may be influenced by its configuration settings (e.g., the threshold for field matching similarity, *sim*). Currently, these settings are determined empirically, and their effectiveness is validated by EmRest's superior performance compared to baselines. Similarly, baseline performance may also depend on their implementations. We obtained the latest stable releases from supplementary materials and used their default settings. While manual adjustments (e.g., providing value dictionaries) could enhance MINER's performance, we intentionally avoided manual tuning for all approaches to ensure fair comparisons, relying solely on the provided OpenAPI specifications. Additionally, all testing approaches involve inherent randomness. To mitigate this impact, each approach was executed 30 times per subject API.

In addition, the availability of error messages might influence EmRest's performance. However, we believe this will not significantly hinder its real-world adoption, since modern REST APIs are expected to provide informative error messages according to best practices and standards [50, 51, 54], as discussed in Section 2. Even if APIs under test lack detailed error messages (or error messages do not explicitly mention input parameters), EmRest can still treat each HTTP status code or each status code-message combination as a distinct error, attempting to infer value assignment strategies most likely to trigger them. We also acknowledge that all testing approaches studied here depend on the availability of OpenAPI specifications. Nevertheless, automated tools exist to generate such specifications from source code [57] or test cases [53]. Thus, we believe these approaches can be extended to test APIs even without provided specifications.

The external threat to validity involves the generalizability of our results, as our experiments were conducted on only 16 subject APIs. To mitigate this, we selected widely recognized benchmark APIs (the first ten APIs in Table 1) commonly used in recent studies [33, 35]. Additionally, we included six APIs from GitLab, a large-scale, real-world enterprise system, to cover more complex testing scenarios. We believe these APIs represent a diverse set of subjects, enhancing the applicability of our findings across various APIs.

8 Related Work

Numerous approaches have been proposed [2, 10, 13, 15, 16, 19, 20, 23, 25, 28, 31, 33, 34, 37, 40–42, 46, 47, 55, 58, 59, 61] to test REST APIs. However, thoroughly examining API behaviors requires careful consideration of complex constraints to generate valid test inputs. These constraints typically include value-specific rules, inter-parameter dependencies, and operation dependencies. Current testing approaches generally extract value-specific rules from fields like *examples* and *default* in the OpenAPI Specification (OAS) [16, 33, 40, 59, 61]. Inter-parameter dependencies, meanwhile, are often embedded in natural-language descriptions. To address this, IDL[17, 27] provides a formal language to specify such dependencies, while NLPtoREST [32] and RestCT [61] leverage NLP techniques for automatic extraction. Operation dependencies, however, are typically implicit and rarely documented. Existing approaches usually rely on heuristic methods that associate input parameters with API responses to infer these dependencies [33, 41, 59, 61].

To enhance testing effectiveness, recent approaches [16, 33, 37, 40–42, 59] incorporate feedback mechanisms based on HTTP status codes to dynamically address constraints. For example, RESTler [16] identifies invalid operation sequences by detecting 400-range status codes. MINER [42] extends this idea, prioritizing sequences producing 200-range responses to explore deeper logic paths. Other approaches utilize graph-based techniques [40, 41, 59] to model dependencies, updating the graph based on observed status codes. ARAT-RL [33] applies reinforcement learning to adjust parameter assignments according to feedback from status codes. Despite their effectiveness

on many APIs, these methods often struggle when operations have multiple constraints, since adjusting inputs to satisfy one constraint may inadvertently violate another.

In contrast to the above approaches, EmRest leverages diagnostic information from error messages to infer constraints. Specifically, it identifies error message fragments to distinguish different constraint violations, then applies a statistical method to determine value-assignment combinations most likely responsible for each violation. Since error message-guided testing has been successfully utilized in fuzzing studies for databases [30], network protocols [18, 52], and web vulnerabilities [22], further exploiting error messages appears promising for enhancing REST API testing effectiveness.

Note that both EmRest and RestCT use combinatorial testing for efficient input space sampling. However, EmRest leverages error messages rather than solely relying on HTTP status codes, enabling it to more precisely infer invalid parameter combinations. Moreover, EmRest obtains value assignment strategies from more diverse sources and incorporates an *Exceptional Testing* phase to detect parameter validation bugs.

Recently, some studies employed external knowledge sources like DBpedia [13] and large language models (LLMs) [34, 37] to generate realistic parameter values satisfying value-specific rules and inter-parameter dependencies. While effective in certain domains, these methods are limited when the knowledge base lacks coverage. Additionally, using LLMs may introduce privacy and confidentiality risks in sensitive contexts. In contrast, EmRest relies solely on OAS and test execution results, making it applicable across a broader range of testing scenarios.

9 Conclusion

This paper presents EmRest, the first black-box REST API testing approach leveraging error message analysis. By analyzing 400-range errors, EmRest identifies and excludes invalid combinations of parameter value assignment strategies. It also mutates valid assignments and utilizes 500-range errors to prioritize operations prone to bugs. Experiments on 16 real-world REST APIs demonstrate that EmRest outperforms existing methods in both test coverage and bug detection.

DATA AVAILABILITY

The source code of EmRest and all experimental scripts are publicly available at our GitHub repository: https://github.com/GIST-NJU/EmRest. A snapshot of this repository, including the exact version used in this paper, is archived on Zenodo [12] to ensure long-term preservation and reproducibility.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (No. 62472209), the Natural Science Foundation of Jiangsu Province (No. BK20221439), and the Primary Research and Development Plan of Jiangsu Province (No. BE2023025-2).

References

[1] 2024. *Built-in String Formats of OpenAPI.* https://swagger.io/docs/specification/data-models/data-types/#string Accessed: 2024-07-02.
[2] 2024. *Dredd.* https://github.com/apiaryio/dredd Accessed: 2024-10-30.
[3] 2024. *FuzzyWuzzy.* https://github.com/seatgeek/fuzzywuzzy Accessed: 2024-07-04.
[4] 2024. *JaCoCo.* https://www.eclemma.org/jacoco/ Accessed: 2024-07-02.
[5] 2024. *JSON.* https://www.json.org/json-en.html Accessed: 2024-07-04.
[6] 2024. *MITMProxy.* https://mitmproxy.org/ Accessed: 2024-07-04.
[7] 2024. *The Official YAML Web Site.* https://yaml.org/ Accessed: 2024-07-04.
[8] 2024. *OpenAPI Specification - Version 3.1.0 | Swagger.* https://swagger.io/specification/ Accessed: 2024-07-02.
[9] 2024. *PICT.* https://github.com/microsoft/pict Accessed: 2024-07-04.

[10] 2024. *Tcases*. https://github.com/Cornutum/tcases Accessed: 2024-10-30.

[11] 2024. *YouTube Data API Specification*. https://api.apis.guru/v2/specs/googleapis.com/youtube/v3/openapi.yaml Accessed: 2024-10-29.

[12] 2025. *Replication package*. https://zenodo.org/records/14940931 Accessed: 2025-02-28.

[13] Juan C. Alonso. 2021. Automated generation of realistic test inputs for web APIs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1666–1668. doi:10.1145/3468264.3473491

[14] Andrea Arcuri. 2018. An experience report on applying software testing academic results in industry: we need usable automated test generation. *Empirical Softw. Engg.* 23, 4 (Aug. 2018), 1959–1981. doi:10.1007/s10664-017-9570-9

[15] Andrea Arcuri. 2019. RESTful API Automated Test Case Generation with EvoMaster. *ACM Trans. Softw. Eng. Methodol.* 28, 1, Article 3 (jan 2019), 37 pages. doi:10.1145/3293455

[16] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RESTler: stateful REST API fuzzing. In *Proceedings of the 41st International Conference on Software Engineering* (Montreal, Quebec, Canada) *(ICSE '19)*. IEEE Press, 748–758. doi:10.1109/ICSE.2019.00083

[17] Saman Barakat, Ana Belén Sánchez, and Sergio Segura. 2023. IDLGen: Automated Code Generation for Inter-parameter Dependencies in Web APIs. In *Service-Oriented Computing: 21st International Conference, ICSOC 2023, Rome, Italy, November 28 – December 1, 2023, Proceedings, Part I* (Rome, Italy). Springer-Verlag, Berlin, Heidelberg, 153–168. doi:10.1007/978-3-031-48421-6_11

[18] Wu Biao, Tang Chaojing, and Zhang Bin. 2021. FFUZZ: A Fast Fuzzing Test Method for Stateful Network Protocol Implementation. In *2021 2nd International Conference on Computer Communication and Network Security (CCNS)*. 75–79. doi:10.1109/CCNS53852.2021.00023

[19] Davide Corradini, Michele Pasqua, and Mariano Ceccato. 2023. Automated Black-Box Testing of Mass Assignment Vulnerabilities in RESTful APIs. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 2553–2564. doi:10.1109/ICSE48619.2023.00213

[20] Davide Corradini, Amedeo Zampieri, Michele Pasqua, Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2022. Automated black-box testing of nominal and error scenarios in RESTful APIs. *Software Testing, Verification and Reliability* 32, 5 (2022), e1808.

[21] Swaroopa Dola, Rory McDaniel, Matthew B. Dwyer, and Mary Lou Soffa. 2024. CIT4DNN: Generating Diverse and Rare Inputs for Neural Networks Using Latent Space Combinatorial Testing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 118, 13 pages. doi:10.1145/3597503.3639106

[22] Adam Doupé, Ludovico Cavedon, Christopher Kruegel, and Giovanni Vigna. 2012. Enemy of the state: a state-aware black-box web vulnerability scanner. In *Proceedings of the 21st USENIX Conference on Security Symposium* (Bellevue, WA) *(Security'12)*. USENIX Association, USA, 26.

[23] Hamza Ed-douibi, Javier Luis Cánovas Izquierdo, and Jordi Cabot. 2018. Automatic Generation of Test Cases for REST APIs: A Specification-Based Approach. *2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC)* (2018), 181–190. doi:10.1109/edoc.2018.00031

[24] Roy Thomas Fielding and Richard N. Taylor. 2000. *Architectural styles and the design of network-based software architectures*. Ph. D. Dissertation. AAI9980887.

[25] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. 2020. Intelligent REST API data fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) *(ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 725–736. doi:10.1145/3368089.3409719

[26] Amid Golmohammadi, Man Zhang, and Andrea Arcuri. 2023. Testing RESTful APIs: A Survey. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 27 (nov 2023), 41 pages. doi:10.1145/3617175

[27] Henk Grent, Aleksei Akimov, and Maurício Aniche. 2021. Automatically Identifying Parameter Constraints in Complex Web APIs: A Case Study at Adyen. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 71–80. doi:10.1109/ICSE-SEIP52600.2021.00016

[28] Zac Hatfield-Dodds and Dmitry Dygalo. 2022. Deriving semantics-aware fuzzers from web API schemas. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 345–346. doi:10.1145/3510454.3528637

[29] Zac Hatfield-Dodds and Dmitry Dygalo. 2022. Deriving semantics-aware fuzzers from web API schemas. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 345–346. doi:10.1145/3510454.3528637

[30] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. DynSQL: Stateful fuzzing for database management systems with complex and valid SQL query generation. In *Proceedings of the 32nd USENIX Conference on Security Symposium*

(Anaheim, CA, USA) *(SEC '23)*. USENIX Association, USA, Article 277, 17 pages.

[31] S. Karlsson, A. Causevic, and D. Sundmark. 2020. QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 131–141. doi:10.1109/ICST46399.2020.00023

[32] Myeongsoo Kim, Davide Corradini, Saurabh Sinha, Alessandro Orso, Michele Pasqua, Rachel Tzoref-Brill, and Mariano Ceccato. 2023. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1232–1243. doi:10.1145/3597926.3598131

[33] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2023. Adaptive REST API Testing with Reinforcement Learning. IEEE Computer Society, 446–458. doi:10.1109/ASE56229.2023.00218

[34] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. 2024. Leveraging Large Language Models to Improve REST API Testing. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) *(ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 37–41. doi:10.1145/3639476.3639769

[35] Myeongsoo Kim, Qi Xin, Saurabh Sinha, and Alessandro Orso. 2022. Automated test generation for REST APIs: no time to rest yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, South Korea) *(ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 289–301. doi:10.1145/3533767.3534401

[36] D.R. Kuhn, D.R. Wallace, and A.M. Gallo. 2004. Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering* 30, 6 (2004), 418–421. doi:10.1109/TSE.2004.24

[37] Tri Le, Thien Tran, Duy Cao, Vy Le, Tien N. Nguyen, and Vu Nguyen. 2024. KAT: Dependency-Aware Automated API Testing with Large Language Models . In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 82–92. doi:10.1109/ICST60714.2024.00017

[38] Manuel Leithner, Andrea Bombarda, Michael Wagner, Angelo Gargantini, and Dimitris E. Simos. 2024. State of the CArt: evaluating covering array generators at scale. *Int. J. Softw. Tools Technol. Transf.* 26, 3 (May 2024), 301–326. doi:10.1007/s10009-024-00745-2

[39] Jinkun Lin, Shaowei Cai, Bing He, Yingjie Fu, Chuan Luo, and Qingwei Lin. 2021. FastCA: An Effective and Efficient Tool for Combinatorial Covering Array Generation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 77–80. doi:10.1109/ICSE-Companion52605.2021.00040

[40] Jiaxian Lin, Tianyu Li, Yang Chen, Guangsheng Wei, Jiadong Lin, Sen Zhang, and Hui Xu. 2023. foREST: A Tree-based Black-box Fuzzing Approach for RESTful APIs. IEEE Computer Society, 695–705. doi:10.1109/ISSRE59848.2023.00023

[41] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: model-based RESTful API testing with execution feedback. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1406–1417. doi:10.1145/3510003.3510133

[42] Chenyang Lyu, Jiacheng Xu, Shouling Ji, Xuhong Zhang, Qinying Wang, Binbin Zhao, Gaoning Pan, Wei Cao, Peng Chen, and Raheem Beyah. 2023. MINER: a hybrid data-driven approach for REST API fuzzing. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) *(SEC '23)*. USENIX Association, USA, Article 253, 18 pages.

[43] Alberto Martin-Lopez, Sergio Segura, Carlos Müller, and Antonio Ruiz-Cortés. 2022. Specification and Automated Analysis of Inter-Parameter Dependencies in Web APIs. *IEEE Transactions on Services Computing* 15, 4 (2022), 2342–2355. doi:10.1109/TSC.2021.3050610

[44] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2019. A Catalogue of Inter-parameter Dependencies in RESTful Web APIs. In *Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings* (Toulouse, France). Springer-Verlag, Berlin, Heidelberg, 399–414. doi:10.1007/978-3-030-33702-5_31

[45] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2019. Test coverage criteria for RESTful web APIs. In *Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation* (Tallinn, Estonia) *(A-TEST 2019)*. Association for Computing Machinery, New York, NY, USA, 15–21. doi:10.1145/3340433.3342822

[46] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2021. RESTest: automated black-box testing of RESTful web APIs. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, Denmark) *(ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 682–685. doi:10.1145/3460319.3469082

[47] A. Giuliano Mirabella, Alberto Martin-Lopez, Sergio Segura, Luis Valencia-Cabrera, and Antonio Ruiz-Cortes. 2021. Deep Learning-Based Prediction of Test Input Validity for RESTful APIs. In *2021 IEEE/ACM Third International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)* (Madrid, Spain, 2021-06). IEEE, 9–16. doi:10.1109/DeepTest52559.2021.00008

[48] Changhai Nie and Hareton Leung. 2011. A survey of combinatorial testing. *ACM Comput. Surv.* 43, 2, Article 11 (Feb. 2011), 29 pages. doi:10.1145/1883612.1883618
[49] Xintao Niu, Changhai Nie, Hareton Leung, Yu Lei, Xiaoyin Wang, Jiaxi Xu, and Yan Wang. 2020. An Interleaving Approach to Combinatorial Testing and Failure-Inducing Interaction Identification. *IEEE Transactions on Software Engineering* 46, 6 (2020), 584–615. doi:10.1109/TSE.2018.2865772
[50] Mark Nottingham and Erik Wilde. 2016. Problem Details for HTTP APIs. RFC 7807. doi:10.17487/RFC7807
[51] Mark Nottingham, Erik Wilde, and Sanjay Dalal. 2023. Problem Details for HTTP APIs. RFC 9457. doi:10.17487/RFC9457
[52] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2020. AFLNET: A Greybox Fuzzer for Network Protocols. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 460–465. doi:10.1109/ICST46399.2020.00062
[53] Postman API Evangelist. 2024. Postman to OpenAPI Collection. https://www.postman.com/api-evangelist/artificial-intelligence/collection/txy0rdd/postman-to-openapi Accessed: 2025-02-11.
[54] RobBagby. 2023. Web API Implementation - Best Practices for Cloud Applications. https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-implementation.
[55] Sergio Segura, José A. Parejo, Javier Troya, and Antonio Ruiz-Cortés. 2018. Metamorphic testing of RESTful web APIs. (2018), 882. doi:10.1145/3180155.3182528
[56] Sunny Shree, Krishna Khadka, Yu Lei, Raghu N. Kacker, and D. Richard Kuhn. 2024. Constructing Surrogate Models in Machine Learning Using Combinatorial Testing and Active Learning. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) *(ASE '24)*. Association for Computing Machinery, New York, NY, USA, 1645–1654. doi:10.1145/3691620.3695532
[57] SpringDoc Contributors. 2024. SpringDoc OpenAPI. https://springdoc.org/ Accessed: 2025-02-11.
[58] Dimitri Stallenberg, Mitchell Olsthoorn, and Annibale Panichella. 2022. Improving test case generation for REST APIs through hierarchical clustering. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering* (Melbourne, Australia) *(ASE '21)*. IEEE Press, 117–128. doi:10.1109/ASE51524.2021.9678586
[59] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTESTGEN: Automated Black-Box Testing of RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)* (2020-10). 142–152. doi:10.1109/ICST46399.2020.00024
[60] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2024. QuCAT: A Combinatorial Testing Tool for Quantum Software. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (Echternach, Luxembourg) *(ASE '23)*. IEEE Press, 2066–2069. doi:10.1109/ASE56229.2023.00062
[61] Huayao Wu, Lixin Xu, Xintao Niu, and Changhai Nie. 2022. Combinatorial testing of RESTful APIs. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 426–437. doi:10.1145/3510003.3510151
[62] Man Zhang and Andrea Arcuri. 2023. Open Problems in Fuzzing RESTful APIs: A Comparison of Tools. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 144 (sep 2023), 45 pages. doi:10.1145/3597205

Received 2024-10-31; accepted 2025-03-31